Jupyter Notebook Execution Report

Name: Amanda Quintino dos Santos

Project Title: BU OMDS

Date: June 28, 2026

Cell 1: ■ Markdown

Homework Week 1

Preparing datasets for analysis

Cell 2: ■ Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import sklearn
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Cell 3: ■ Code

```
predict_conversion_df = pd.read_csv("digital_marketing_campaign_dataset.csv")
google_ads_df = pd.read_csv("GoogleAds_DataAnalytics_Sales_Uncleaned.csv")
marketing_products_df = pd.read_csv("marketing_and_product_performance.csv")
```

Cell 4: ■ Markdown

Cleaning

Cell 5: ■ Code

```
# Identify missing values for predict_conversion_df
print("Missing values before imputation:\\n", predict_conversion_df.isnull().sum())
```

Output:

```
Missing values before imputation:\n CustomerID            0
Age                   0
Gender                0
Income                0
CampaignChannel       0
CampaignType          0
AdSpend               0
ClickThroughRate      0
ConversionRate        0
WebsiteVisits         0
PagesPerVisit         0
TimeOnSite            0
SocialShares          0
EmailOpens            0
EmailClicks           0
PreviousPurchases     0
LoyaltyPoints         0
AdvertisingPlatform   0
AdvertisingTool       0
Conversion            0
dtype: int64
```

Cell 6: ■ Code

```
# Clean the strings
for col in ['Cost', 'Sale_Amount']:
google_ads_df[col] = google_ads_df[col].astype(str).str.replace(r'[\$,]', '',
regex=True)
# Convert 'nan' strings back to actual NaNs before making numeric
google_ads_df[col] = pd.to_numeric(google_ads_df[col], errors='coerce')
```

```
# Impute with mean
columns_to_impute = ['Cost', 'Sale_Amount']
google_ads_df[columns_to_impute] =
google_ads_df[columns_to_impute].fillna(google_ads_df[columns_to_impute].mean())
```

```
# Check remaining missing values
print("Remaining missing values in google_ads_df:\n", google_ads_df.isnull().sum())
```

Output:

```
Remaining missing values in google_ads_df:
 Ad_ID                0
Campaign_Name         0
Clicks              112
Impressions          54
Cost                  0
Leads                48
Conversions          74
Conversion Rate     626
Sale_Amount           0
Ad_Date               0
Location              0
Device                0
Keyword               0
dtype: int64

[STDERR]
&lt;string&gt;:3: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but whe
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/ind

&lt;string&gt;:5: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but whe
A typical example is when you are setting values in a column of a DataFrame, like:
```

```
df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/ind

&lt;string&gt;:3: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but whe
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/ind

&lt;string&gt;:5: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but whe
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/ind

c:\Users\santo\OneDrive\Documents\GitHub\Learning\.venv\Lib\site-packages\pandas\core\generic.py:
You are setting values through chained assignment. Currently this works in certain cases, but whe
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/ind

  result[k] = res_k
&lt;string&gt;:9: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but whe
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/ind
```

Cell 7: ■ Code

```
# Identify missing values in marketing_products_df

print("Missing values in marketing_products_df:\\n",
marketing_products_df.isnull().sum())
```

Output:

```
Missing values in marketing_products_df:\n Campaign_ID                          0
Product_ID                           0
Budget                               0
Clicks                               0
Conversions                          0
Revenue_Generated                    0
ROI                                  0
Customer_ID                          0
Subscription_Tier                    0
Subscription_Length                  0
Flash_Sale_ID                        0
Discount_Level                       0
Units_Sold                           0
Bundle_ID                            0
Bundle_Price                         0
Customer_Satisfaction_Post_Refund    0
Common_Keywords                      0
dtype: int64
```

Cell 8: ■ Markdown

Predict_Conversion_df analysis

Cell 9: ■ Code

```
# Select numeric features for X, and 'Conversion' as the target variable y

# Dropping ID columns and target variables from features

numeric_features = predict_conversion_df.select_dtypes(include=[np.number]).drop(co
lumns=['CustomerID', 'Conversion', 'ConversionRate'], errors='ignore')
```

```
X = numeric_features
```

```
y = predict_conversion_df['Conversion']
```

```
# Train-test split (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```
# Generate Polynomial Features (degree=2)
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
```

```
# Train the Linear Regression model on polynomial features
lr_poly = LinearRegression()
lr_poly.fit(X_train_poly, y_train)
```

```
# Make predictions and evaluate
y_pred = lr_poly.predict(X_test_poly)
```

```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

```
print(f"Polynomial Regression (Degree 2) Results:")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"R^2 Score: {r2:.4f}")
```

Output:

```
Polynomial Regression (Degree 2) Results:
Mean Squared Error (MSE): 0.0902
R^2 Score: 0.1534
```

Cell 10: ■ Code

```
# Generate Interaction Features (only interactions, no squared terms)
interaction = PolynomialFeatures(degree=2, interaction_only=True,
include_bias=False)
X_train_int = interaction.fit_transform(X_train)
X_test_int = interaction.transform(X_test)
```

```
# Get the feature names for interactions
feature_names = interaction.get_feature_names_out(X.columns)
```

```
# Train the Linear Regression model
lr_int = LinearRegression()
lr_int.fit(X_train_int, y_train)
```

```
# Evaluate and look at coefficients
y_pred_int = lr_int.predict(X_test_int)
mse_int = mean_squared_error(y_test, y_pred_int)
r2_int = r2_score(y_test, y_pred_int)
```

```
print(f"Interaction-Only Regression Results:")
print(f"Mean Squared Error (MSE): {mse_int:.4f}")
print(f"R^2 Score: {r2_int:.4f}\\n")
```

```
# Combine feature names with their coefficients
coef_df = pd.DataFrame({
'Feature': feature_names,
'Coefficient': lr_int.coef_
})
```

```
# Filter out individual features to only look at interactions
interaction_coefs = coef_df[coef_df['Feature'].str.contains(' ')]
```

```
# Display top 10 most impactful interactions (by absolute coefficient value)
interaction_coefs['Abs_Coef'] = interaction_coefs['Coefficient'].abs()
top_interactions = interaction_coefs.sort_values(by='Abs_Coef',
ascending=False).head(10)
```

```
print("Top 10 Most Impactful Interaction Terms:")
print(top_interactions[['Feature', 'Coefficient']])
```

Output:

```
Interaction-Only Regression Results:
Mean Squared Error (MSE): 0.0940
R^2 Score: 0.1173\n
Top 10 Most Impactful Interaction Terms:
                              Feature  Coefficient
43      ClickThroughRate PagesPerVisit    -0.074534
48  ClickThroughRate PreviousPurchases    -0.065516
47        ClickThroughRate EmailClicks    -0.045399
```

```
44         ClickThroughRate TimeOnSite    -0.043712
46         ClickThroughRate EmailOpens    -0.032472
75       EmailClicks PreviousPurchases    -0.001738
45       ClickThroughRate SocialShares    -0.001709
60           PagesPerVisit EmailClicks    -0.001569
14                Age ClickThroughRate    -0.001336
57            PagesPerVisit TimeOnSite    -0.001326

[STDERR]
&lt;string&gt;:32: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but whe
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/ind

&lt;string&gt;:32: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/ind
```

Cell 11: ■ Code

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm
```

```
# Select independent variables
vif_features = predict_conversion_df.select_dtypes(include=[np.number]).drop(column
s=['CustomerID', 'Conversion', 'ConversionRate'], errors='ignore')
```

```
# Add a constant term to the independent variables
X_vif = sm.add_constant(vif_features)
```

```
# Calculate VIF for each feature
vif_data = pd.DataFrame()
vif_data["Feature"] = X_vif.columns
vif_data["VIF"] = [variance_inflation_factor(X_vif.values, i) for i in
range(X_vif.shape[1])]
```

```
# Display VIF excluding the constant, sorted by highest multicollinearity
print("Variance Inflation Factor (VIF):")
vif_result = vif_data[vif_data["Feature"] != "const"].sort_values(by="VIF", ascending=False)
print(vif_result)
```

```
print("\\nNote: VIF &gt; 5 or 10 suggests high multicollinearity.")
```

Output:

```
Variance Inflation Factor (VIF):
              Feature       VIF
7          TimeOnSite  1.001574
4    ClickThroughRate  1.001515
5       WebsiteVisits  1.001464
1                 Age  1.001138
8        SocialShares  1.001137
6       PagesPerVisit  1.001047
12      LoyaltyPoints  1.001012
11  PreviousPurchases  1.000974
3             AdSpend  1.000974
2              Income  1.000854
9          EmailOpens  1.000745
10        EmailClicks  1.000308
\nNote: VIF &gt; 5 or 10 suggests high multicollinearity.
```

Cell 12: ■ Code

```
# Identify categorical columns to encode
categorical_cols = predict_conversion_df.select_dtypes(include=['object', 'category']).columns.tolist()
categorical_cols = [col for col in categorical_cols if col not in ['CustomerID', 'Conversion']]
```

```
# Create feature matrix X by dropping target and IDs
X_all = predict_conversion_df.drop(columns=['CustomerID', 'Conversion', 'ConversionRate'], errors='ignore')
y_target = predict_conversion_df['Conversion']
```

```
# One-hot encode the categorical features
X_encoded = pd.get_dummies(X_all, columns=categorical_cols, drop_first=True)
```

```
# Train-test split
X_train_enc, X_test_enc, y_train_enc, y_test_enc = train_test_split(X_encoded,
y_target, test_size=0.2, random_state=42)
```

```
# Train the Linear Regression model
lr_ohe = LinearRegression()
lr_ohe.fit(X_train_enc, y_train_enc)
```

```
# Make predictions and evaluate
y_pred_enc = lr_ohe.predict(X_test_enc)
mse_enc = mean_squared_error(y_test_enc, y_pred_enc)
r2_enc = r2_score(y_test_enc, y_pred_enc)
```

```
print(f"Linear Regression with One-Hot Encoded Features Results:")
print(f"Mean Squared Error (MSE): {mse_enc:.4f}")
print(f"R^2 Score: {r2_enc:.4f}")
```

Output:

```
Linear Regression with One-Hot Encoded Features Results:
Mean Squared Error (MSE): 0.0948
R^2 Score: 0.1103
```

Cell 13: ■ Markdown

Google_Ads_df Analysis

Cell 14: ■ Code

```
# Clean google_ads_df by dropping rows with missing targets and imputing missing
features
google_ads_df = google_ads_df.dropna(subset=['Conversions'])
numeric_cols_ga = google_ads_df.select_dtypes(include=[np.number]).columns
google_ads_df[numeric_cols_ga] =
google_ads_df[numeric_cols_ga].fillna(google_ads_df[numeric_cols_ga].mean())
```

```
# Drop non-predictive or target-related variables
```

```
numeric_features_google =
google_ads_df.select_dtypes(include=[np.number]).drop(columns=['Conversions',
'Conversion Rate'], errors='ignore')
```

```
X_ga = numeric_features_google

y_ga = google_ads_df['Conversions']
```

```
# Train-test split (80% training, 20% testing)

X_train_ga, X_test_ga, y_train_ga, y_test_ga = train_test_split(X_ga, y_ga,
test_size=0.2, random_state=42)
```

```
# Generate Polynomial Features (degree=2)

poly_ga = PolynomialFeatures(degree=2, include_bias=False)

X_train_poly_ga = poly_ga.fit_transform(X_train_ga)

X_test_poly_ga = poly_ga.transform(X_test_ga)
```

```
# Train the Linear Regression model on polynomial features

lr_poly_ga = LinearRegression()

lr_poly_ga.fit(X_train_poly_ga, y_train_ga)
```

```
# Make predictions and evaluate

y_pred_ga = lr_poly_ga.predict(X_test_poly_ga)
```

```
mse_ga = mean_squared_error(y_test_ga, y_pred_ga)

r2_ga = r2_score(y_test_ga, y_pred_ga)
```

```
print(f"Polynomial Regression (Degree 2) on google_ads_df Results:")

print(f"Mean Squared Error (MSE): {mse_ga:.4f}")

print(f"R^2 Score: {r2_ga:.4f}")
```

Output:

```
Polynomial Regression (Degree 2) on google_ads_df Results:

Mean Squared Error (MSE): 5.1717

R^2 Score: -0.0166

[STDERR]

c:\Users\santo\OneDrive\Documents\GitHub\Learning\.venv\Lib\site-packages\pandas\core\generic.py:

You are setting values through chained assignment. Currently this works in certain cases, but whe

A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value
```

```
Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/ind

  result[k] = res_k
```

Cell 15: ■ Code

```
# Generate Interaction Features for google_ads_df
interaction_ga = PolynomialFeatures(degree=2, interaction_only=True,
include_bias=False)
X_train_int_ga = interaction_ga.fit_transform(X_train_ga)
X_test_int_ga = interaction_ga.transform(X_test_ga)
```

```
# Get the feature names for interactions
feature_names_ga = interaction_ga.get_feature_names_out(X_ga.columns)
```

```
# Train the Linear Regression model
lr_int_ga = LinearRegression()
lr_int_ga.fit(X_train_int_ga, y_train_ga)
```

```
# Evaluate coefficients
y_pred_int_ga = lr_int_ga.predict(X_test_int_ga)
mse_int_ga = mean_squared_error(y_test_ga, y_pred_int_ga)
r2_int_ga = r2_score(y_test_ga, y_pred_int_ga)
```

```
print(f"Interaction-Only Regression on google_ads_df Results:")
print(f"Mean Squared Error (MSE): {mse_int_ga:.4f}")
print(f"R^2 Score: {r2_int_ga:.4f}\\n")
```

```
# Combine feature names with their coefficients
coef_df_ga = pd.DataFrame({
'Feature': feature_names_ga,
'Coefficient': lr_int_ga.coef_
})
```

```
# Filter out individual features to only look at interactions
interaction_coefs_ga = coef_df_ga[coef_df_ga['Feature'].str.contains(' ')].copy()
```

```
# Display top 10 most impactful interactions (by absolute coefficient value)
interaction_coefs_ga['Abs_Coef'] = interaction_coefs_ga['Coefficient'].abs()
```

```
top_interactions_ga = interaction_coefs_ga.sort_values(by='Abs_Coef',
ascending=False).head(10)
```

```
print("Top 10 Most Impactful Interaction Terms in google_ads_df:")
print(top_interactions_ga[['Feature', 'Coefficient']])
```

Output:

```
Interaction-Only Regression on google_ads_df Results:
Mean Squared Error (MSE): 5.1610
R^2 Score: -0.0145\n
Top 10 Most Impactful Interaction Terms in google_ads_df:
                    Feature   Coefficient
12               Cost Leads -1.670977e-04
6                Clicks Cost -1.666654e-04
7               Clicks Leads -1.431119e-04
14         Leads Sale_Amount -3.767286e-05
10         Impressions Leads -1.159205e-05
13          Cost Sale_Amount -1.004500e-05
8         Clicks Sale_Amount  2.851685e-06
5         Clicks Impressions -1.883470e-06
9           Impressions Cost  4.584465e-07
11  Impressions Sale_Amount  4.326796e-07

[STDERR]
&lt;string&gt;:32: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but whe
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/ind
```

Cell 16: ■ Code

```
# Calculate VIF for google_ads_df features to detect multicollinearity
```

```
# Select independent numeric variables - exclude targets and uninformative IDs
```

```
vif_features_ga =
google_ads_df.select_dtypes(include=[np.number]).drop(columns=['Conversions',
'Conversion Rate'], errors='ignore')
```

```
# Add a constant term to the independent variables (required for VIF calculation)
X_vif_ga = sm.add_constant(vif_features_ga)
```

```
# Calculate VIF for each feature
vif_data_ga = pd.DataFrame()
vif_data_ga["Feature"] = X_vif_ga.columns
vif_data_ga["VIF"] = [variance_inflation_factor(X_vif_ga.values, i) for i in
range(X_vif_ga.shape[1])]
```

```
# Display VIF excluding the constant, sorted by highest multicollinearity
print("Variance Inflation Factor (VIF) for google_ads_df:")
vif_result_ga = vif_data_ga[vif_data_ga["Feature"] !=
"const"].sort_values(by="VIF", ascending=False)
print(vif_result_ga)
```

```
print("\\nNote: VIF &gt; 5 or 10 suggests high multicollinearity.")
```

Output:

```
Variance Inflation Factor (VIF) for google_ads_df:
       Feature       VIF
1       Clicks  1.005553
4        Leads  1.004935
5  Sale_Amount  1.002104
2  Impressions  1.001564
3         Cost  1.001391
\nNote: VIF &gt; 5 or 10 suggests high multicollinearity.
```

Cell 17: ■ Code

```
# Identify categorical columns to encode in google_ads_df
categorical_cols_ga = google_ads_df.select_dtypes(include=['object',
'category']).columns.tolist()
# Removing 'Ad_ID' and 'Ad_Date' to prevent high cardinality issues
categorical_cols_ga = [col for col in categorical_cols_ga if col not in ['Ad_ID',
'Ad_Date']]
```

```
# Drop non-predictive/target identifiers
X_all_ga = google_ads_df.drop(columns=['Ad_ID', 'Ad_Date', 'Conversions',
'Conversion Rate'], errors='ignore')
y_target_ga = google_ads_df['Conversions']
```

```
# One-hot encode the qualifying categorical features
X_encoded_ga = pd.get_dummies(X_all_ga, columns=categorical_cols_ga,
drop_first=True)
```

```
# Train-test split (80% / 20%)
X_train_enc_ga, X_test_enc_ga, y_train_enc_ga, y_test_enc_ga = train_test_split(
X_encoded_ga, y_target_ga, test_size=0.2, random_state=42
)
```

```
# Train a simple Linear Regression model
lr_ohe_ga = LinearRegression()
lr_ohe_ga.fit(X_train_enc_ga, y_train_enc_ga)
```

```
y_pred_enc_ga = lr_ohe_ga.predict(X_test_enc_ga)
mse_enc_ga = mean_squared_error(y_test_enc_ga, y_pred_enc_ga)
r2_enc_ga = r2_score(y_test_enc_ga, y_pred_enc_ga)
```

```
print(f"Linear Regression with One-Hot Encoded Features on google_ads_df Results:")
print(f"Mean Squared Error (MSE): {mse_enc_ga:.4f}")
print(f"R^2 Score: {r2_enc_ga:.4f}")
```

Output:

```
Linear Regression with One-Hot Encoded Features on google_ads_df Results:
Mean Squared Error (MSE): 5.0822
R^2 Score: 0.0010
```

Cell 18: ■ Markdown

Marketing_Products_df analysis

Cell 19: ■ Code

```
# Select numeric features
```

```
numeric_features_mp =
marketing_products_df.select_dtypes(include=[np.number]).drop(
columns=['Campaign_ID', 'Product_ID', 'Customer_ID', 'Flash_Sale_ID', 'Bundle_ID',
'Conversions', 'Revenue_Generated', 'ROI'],
errors='ignore'
)
```

```
X_mp = numeric_features_mp
y_mp = marketing_products_df['Conversions']
```

```
# Train-test split (80% training, 20% testing)
X_train_mp, X_test_mp, y_train_mp, y_test_mp = train_test_split(X_mp, y_mp,
test_size=0.2, random_state=42)
```

```
# Generate Polynomial Features (degree=2)
poly_mp = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly_mp = poly_mp.fit_transform(X_train_mp)
X_test_poly_mp = poly_mp.transform(X_test_mp)
```

```
# Train the Linear Regression model on polynomial features
lr_poly_mp = LinearRegression()
lr_poly_mp.fit(X_train_poly_mp, y_train_mp)
```

```
# Make predictions and evaluate
y_pred_mp = lr_poly_mp.predict(X_test_poly_mp)
```

```
mse_mp = mean_squared_error(y_test_mp, y_pred_mp)
r2_mp = r2_score(y_test_mp, y_pred_mp)
```

```
print(f"Polynomial Regression (Degree 2) on marketing_products_df Results:")
print(f"Mean Squared Error (MSE): {mse_mp:.4f}")
print(f"R^2 Score: {r2_mp:.4f}")
```

Output:

```
Polynomial Regression (Degree 2) on marketing_products_df Results:
Mean Squared Error (MSE): 85066.9266
R^2 Score: -0.0048
```

Cell 20: ■ Code

```
# Generate Interaction Features (only interactions, no squared terms)
interaction_mp = PolynomialFeatures(degree=2, interaction_only=True,
include_bias=False)
X_train_int_mp = interaction_mp.fit_transform(X_train_mp)
X_test_int_mp = interaction_mp.transform(X_test_mp)
```

```
# Get the feature names for interactions
feature_names_mp = interaction_mp.get_feature_names_out(X_mp.columns)
```

```
# Train the Linear Regression model
lr_int_mp = LinearRegression()
lr_int_mp.fit(X_train_int_mp, y_train_mp)
```

```
# Evaluate coefficients
y_pred_int_mp = lr_int_mp.predict(X_test_int_mp)
mse_int_mp = mean_squared_error(y_test_mp, y_pred_int_mp)
r2_int_mp = r2_score(y_test_mp, y_pred_int_mp)
```

```
print(f"Interaction-Only Regression on marketing_products_df Results:")
print(f"Mean Squared Error (MSE): {mse_int_mp:.4f}")
print(f"R^2 Score: {r2_int_mp:.4f}\\n")
```

```
# Combine feature names with their coefficients
coef_df_mp = pd.DataFrame({
'Feature': feature_names_mp,
'Coefficient': lr_int_mp.coef_
})
```

```
# Filter out individual features to only have interactions
interaction_coefs_mp = coef_df_mp[coef_df_mp['Feature'].str.contains(' ')].copy()
```

```
# Display top 10 most impactful interactions (by absolute coefficient value)
interaction_coefs_mp['Abs_Coef'] = interaction_coefs_mp['Coefficient'].abs()
top_interactions_mp = interaction_coefs_mp.sort_values(by='Abs_Coef',
ascending=False).head(10)
```

```
print("Top 10 Most Impactful Interaction Terms in marketing_products_df:")
print(top_interactions_mp[['Feature', 'Coefficient']])
```

Output:

```
Interaction-Only Regression on marketing_products_df Results:
Mean Squared Error (MSE): 84924.4616
R^2 Score: -0.0031\n
Top 10 Most Impactful Interaction Terms in marketing_products_df:
                                            Feature  Coefficient
24   Discount_Level Customer_Satisfaction_Post_Refund     0.166476
21  Subscription_Length Customer_Satisfaction_Post...     0.158133
26       Units_Sold Customer_Satisfaction_Post_Refund     0.082755
27     Bundle_Price Customer_Satisfaction_Post_Refund    -0.011824
18                 Subscription_Length Discount_Level    -0.009723
19                     Subscription_Length Units_Sold     0.005765
22                          Discount_Level Units_Sold     0.001339
23                       Discount_Level Bundle_Price     0.001075
20                   Subscription_Length Bundle_Price    -0.000890
17           Clicks Customer_Satisfaction_Post_Refund     0.000753
[STDERR]
&lt;string&gt;:32: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but whe
A typical example is when you are setting values in a column of a DataFrame, like:
df["col"][row_indexer] = value
Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/ind
```

Cell 21: ■ Code

```
# Calculate VIF for marketing_products_df features to detect multicollinearity
```

```
# Select independent numeric variables - exclude targets and IDs
vif_features_mp = marketing_products_df.select_dtypes(include=[np.number]).drop(
columns=['Campaign_ID', 'Product_ID', 'Customer_ID', 'Flash_Sale_ID', 'Bundle_ID',
'Conversions', 'Revenue_Generated', 'ROI'],
errors='ignore'
)
```

```
# Add a constant term to the independent variables
```

```
X_vif_mp = sm.add_constant(vif_features_mp)
```

```
# Calculate VIF for each feature
vif_data_mp = pd.DataFrame()
vif_data_mp["Feature"] = X_vif_mp.columns
vif_data_mp["VIF"] = [variance_inflation_factor(X_vif_mp.values, i) for i in range(X_vif_mp.shape[1])]
```

```
# Display VIF excluding the constant, sorted by highest multicollinearity
print("Variance Inflation Factor (VIF) for marketing_products_df:")
vif_result_mp = vif_data_mp[vif_data_mp["Feature"] != "const"].sort_values(by="VIF", ascending=False)
print(vif_result_mp)
```

```
print("\\nNote: VIF &gt; 5 or 10 suggests high multicollinearity.")
```

Output:

```
Variance Inflation Factor (VIF) for marketing_products_df:
                            Feature       VIF
1                            Budget  1.000735
5                        Units_Sold  1.000610
2                            Clicks  1.000545
4                     Discount_Level  1.000436
3                Subscription_Length  1.000426
7  Customer_Satisfaction_Post_Refund  1.000379
6                       Bundle_Price  1.000210
\nNote: VIF &gt; 5 or 10 suggests high multicollinearity.
```

Cell 22: ■ Code

```
# Identify categorical columns to encode in marketing_products_df
categorical_cols_mp = marketing_products_df.select_dtypes(include=['object', 'category']).columns.tolist()
```

```
# Define identifiers to be excluded
identifiers_mp = ['Campaign_ID', 'Product_ID', 'Customer_ID', 'Flash_Sale_ID', 'Bundle_ID']
```

```
# Remove identifiers from categorical list
```

```
categorical_cols_mp = [col for col in categorical_cols_mp if col not in
identifiers_mp]
```

```
# Drop target leaks and identifiers from the feature matrix
features_to_drop_mp = identifiers_mp + ['Conversions', 'Revenue_Generated', 'ROI']
X_all_mp = marketing_products_df.drop(columns=features_to_drop_mp, errors='ignore')
y_target_mp = marketing_products_df['Conversions']
```

```
# One-hot encode the qualifying categorical features
X_encoded_mp = pd.get_dummies(X_all_mp, columns=categorical_cols_mp,
drop_first=True)
```

```
# Train-test split (80% / 20%)
X_train_enc_mp, X_test_enc_mp, y_train_enc_mp, y_test_enc_mp = train_test_split(
X_encoded_mp, y_target_mp, test_size=0.2, random_state=42
)
```

```
# Train a simple Linear Regression model
lr_ohe_mp = LinearRegression()
lr_ohe_mp.fit(X_train_enc_mp, y_train_enc_mp)
```

```
# Make predictions and evaluate performance
y_pred_enc_mp = lr_ohe_mp.predict(X_test_enc_mp)
mse_enc_mp = mean_squared_error(y_test_enc_mp, y_pred_enc_mp)
r2_enc_mp = r2_score(y_test_enc_mp, y_pred_enc_mp)
```

```
print(f"Linear Regression with One-Hot Encoded Features on marketing_products_df
Results:")
print(f"Mean Squared Error (MSE): {mse_enc_mp:.4f}")
print(f"R^2 Score: {r2_enc_mp:.4f}")
```

Output:

```
Linear Regression with One-Hot Encoded Features on marketing_products_df Results:
Mean Squared Error (MSE): 84622.7355
R^2 Score: 0.0004
```